Project Name:	**Toothwise — AI-Powered Dental Screening System**
Module Name:	**The Whole System**
System/Component:	**Full Stack (FastAPI + React + YOLO + scikit-learn KNN)**
Created By:	**Arreola, Tubillas, Ponce, Sales, and Magpantay**
Date of Creation:	**08 / 25 / 2026**
Date of Revision:	**09 / 23 / 2026**

TOOTHWISE

TEST CASE MATRIX

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_LOGIN_001	Verify dentist login with valid credentials	Enter valid User Name and valid Password	Account is already logged in	1. Enter User Name 2. Enter Password 3. Click 'Login' button	Valid Email: admin@codismile.com Valid Password: CodiSmile@2025	Successfully login	Toothwise Dentist Dashboard	Toothwise Dentist Dashboard	**Pass**
TC_LOGIN_002	Verify dentist login with invalid password	Enter valid User Name and invalid Password	Account is already logged in	1. Enter User Name 2. Enter Password 3. Click 'Login' button	Valid Email: admin@codismile.com Invalid Password: wrongpassword	A message "The email and password do not match our records" is shown	Toothwise login page	A message "The email and password do not match our records" is shown	**Pass**
TC_LOGIN_003	Verify dentist login with non-existent email	Enter invalid User Name and valid Password	Account is already logged in	1. Enter User Name 2. Enter Password 3. Click 'Login' button	Invalid Email: nobody@example.com Valid Password: anything	A message "The email and password do not match our records" is shown	Toothwise login page	A message "The email and password do not match our records" is shown	**Pass**
TC_LOGIN_004	Verify patient self-registration	Enter valid patient details and register	None	1. Enter First Name 2. Enter Last Name 3. Enter Email 4. Enter Password 5. Click 'Register' button	Valid Name: Test Patient Valid Email: patient@test.com Valid Password: Secure123!	Successfully registered; JWT token returned	Toothwise login page	Patient account created; JWT returned	**Pass**
TC_LANDING_PAGE_001	Successfully login to Toothwise and land on Dentist Dashboard	Be able to see and interact with Dentist Dashboard	TC_LOGIN_001	1. Login with valid credentials 2. Observe Dentist Dashboard	Valid User Name Valid Password	See Dentist Dashboard page	Toothwise Dentist Dashboard	Toothwise Dentist Dashboard	**Pass**
TC_PATIENT_001	Verify admin can create a patient	Create patient with valid details	Admin is logged in	1. Click 'Patients' menu 2. Click 'Add Patient' 3. Fill form 4. Click 'Save'	Valid Name: Maria Clara Valid Birthdate: 1995-03-20 Valid Gender: Female	Patient created successfully; 201 Returned	Toothwise Patients list	Patient created successfully	**Pass**
TC_PATIENT_002	Verify patient list is displayed	View list of all patients	Admin is logged in; at least one patient exists	1. Click 'Patients' menu 2. Observe patient list	No input required	Patient list displayed with >= 1 entry	Toothwise Patients list	Patient list displayed	**Pass**
TC_PATIENT_003	Verify patient search by name	Search patient by name substring	Patient 'Juan Dela Cruz' exists	1. Click 'Patients' menu 2. Enter 'Juan' in search box 3. Observe results	Search term: Juan	Results include matching patient	Toothwise Patients list	Matching patient found	**Pass**
TC_PATIENT_004	Verify patient details can be updated	Update patient name	Patient seeded	1. Click patient row 2. Click 'Edit' 3. Change First Name 4. Click 'Save'	New First Name: Juan Pedro	Patient name updated to 'Juan Pedro Cruz'	Toothwise Patient detail page	Name updated	**Pass**
TC_PATIENT_005	Verify admin can delete a patient	Delete an existing patient	Patient seeded	1. Click patient row 2. Click 'Delete' 3. Confirm deletion	Patient ID from seed	Patient removed; 204 No Content	Toothwise Patients list	Patient removed	**Pass**
TC_SCREENING_001	Verify screening creation with mock model	Upload intraoral image for AI screening	Admin logged in; patient exists; mock model enabled	1. Go to patient detail 2. Click 'New Screening' 3. Upload image 4. Click 'Analyze'	Valid PNG image (64x64)	Screening created; risk_level in (LOW,MEDIUM,HIGH); 0 <= risk_score <= 100	Toothwise Screening detail page	Screening created with risk level and score	**Pass**
TC_SCREENING_002	Verify screening has fused detections	View fused detection set for a screening	Screening exists	1. Go to Screening detail 2. Observe detections list	Screening ID	Detections include at least one with detection_set='fused'	Toothwise Screening detail page	Fused detections present	**Pass**
TC_SCREENING_003	Verify screening has prevention plan	View KNN-generated recommendations	Screening exists	1. Go to Screening detail 2. Scroll to Recommendations section	Screening ID	Recommendations array non-empty; each item has 'text' field	Toothwise Screening detail page	Recommendations displayed	**Pass**
TC_SCREENING_004	Verify YOLOv8 vs YOLOv12 comparison	View model comparison for a screening	Screening exists	1. Go to Screening detail 2. Click 'Compare Models'	Screening ID	Response has yolov8, yolov12, fused_count, agreement_count	Toothwise Model Comparison page	Comparison data displayed	**Pass**
TC_SCREENING_005	Verify dentist can add clinical note	Add a clinical note to a screening	Screening exists; dentist logged in	1. Go to Screening detail 2. Click 'Add Note' 3. Enter note text 4. Click 'Save'	Note body: Mild gingivitis observed	Note saved; 201 Created; body matches input	Toothwise Screening detail page	Note saved and displayed	**Pass**
TC_SCREENING_006	Verify unauthenticated screening access is blocked	Access screening without login	None	1. Navigate to /api/screenings/1 without token	No token	401 or 403 returned	Toothwise login page	Access denied	**Pass**
TC_SCREENING_007	Verify screening for non-existent patient returns 404	Create screening for patient that does not exist	Admin logged in	1. POST /api/patients/99999/screening with image	Non-existent patient ID	404 Not Found	No screening created	404 Not Found	**Pass**
TC_SCREENING_008	Verify admin can delete a screening	Delete an existing screening	Screening exists; admin logged in	1. Go to Screening detail 2. Click 'Delete' 3. Confirm	Screening ID	204 No Content; screening removed	Toothwise Patient detail page	Screening removed	**Pass**

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_DENTIST_001	Verify admin can list all dentists	View dentist directory	Admin logged in; >= 2 dentists exist	1. Click 'Dentists' menu 2. Observe list	No input required	Dentist list shows all dentists	Toothwise Dentists list	All dentists listed	Pass
TC_DENTIST_002	Verify admin can create a dentist	Create a new dentist account	Admin logged in	1. Click 'Add Dentist' 2. Fill form 3. Click 'Save'	Valid Name: New Doctor Valid Email: new@codismile.com Valid Password: NewPass123!	Dentist created; 201 Returned	Toothwise Dentists list	Dentist created	Pass
TC_DENTIST_003	Verify duplicate email is rejected	Attempt to create dentist with existing email	Dentist with same email exists	1. Click 'Add Dentist' 2. Fill form with duplicate email 3. Click 'Save'	Duplicate Email: dentist@test.com	409 Conflict returned	Toothwise Dentists list	409 Conflict	Pass
TC_DENTIST_004	Verify admin can delete a dentist	Delete a non-admin dentist	Non-admin dentist seeded	1. Click dentist row 2. Click 'Delete' 3. Confirm	Dentist ID	204 No Content; dentist removed	Toothwise Dentists list	Dentist removed	Pass
TC_DENTIST_005	Verify admin cannot delete themselves	Attempt to delete own account	Admin logged in	1. Attempt DELETE /api/dentists/{own_id}	Admin's own ID	400 Bad Request	Account preserved	400 Bad Request	Pass
TC_DENTIST_006	Verify non-admin cannot change own role	Attempt privilege escalation	Non-admin dentist logged in	1. PUT /api/dentists/{id} with role='admin'	{"role":"admin"}	403 Forbidden	Role unchanged	403 Forbidden	Pass
TC_DENTIST_007	Verify non-admin cannot change own email	Attempt to change own email	Non-admin dentist logged in	1. PUT /api/dentists/{id} with new email	{"email":"hacked@evil.com"}	403 Forbidden	Email unchanged	403 Forbidden	Pass
TC_DENTIST_008	Verify non-admin cannot deactivate themselves	Attempt self-deactivation	Non-admin dentist logged in	1. PUT /api/dentists/{id} with is_active=false	{"is_active":false}	403 Forbidden	Account still active	403 Forbidden	Pass
TC_DENTIST_009	Verify non-admin can update own name	Update profile name	Non-admin dentist logged in	1. PUT /api/dentists/{id} with new name	{"first_name":"Updated","last_name":"Name"}	200 OK; name updated	Name updated in DB	200 OK; name=Updated Name	Pass
TC_DENTIST_010	Verify admin can change dentist role	Promote/demote another dentist	Admin logged in; target dentist seeded	1. PUT /api/dentists/{id} with role='admin'	{"role":"admin"}	200 OK; role updated	Role changed in DB	200 OK; role=admin	Pass
TC_DENTIST_011	Verify admin can reset dentist password	Reset another dentist's password	Admin logged in; target dentist seeded	1. PUT /api/dentists/{id}/password with new password	{"new_password":"ResetPass123!"}	200 OK; new password works on login	Password updated	200; login with new password succeeds	Pass
TC_DENTIST_012	Verify non-admin cannot reset own password via endpoint	Attempt password reset without admin	Non-admin dentist logged in	1. PUT /api/dentists/{id}/password with new password	{"new_password":"HackedPass!"}	403 Forbidden	Password unchanged	403 Forbidden	Pass
TC_RISK_001	Verify healthy detection scores LOW	Compute risk for healthy_tooth only	None	1. Call compute_risk with healthy_tooth detection	[{"label":"healthy_tooth","confidence":0.9}]	score=0, level='LOW'	None	score=0, level=LOW	Pass
TC_RISK_002	Verify cavity detection scores HIGH	Compute risk for high-severity finding	None	1. Call compute_risk with cavity detection	[{"label":"cavity","confidence":1.0}]	level='HIGH', score >= 70	None	level=HIGH, score>=70	Pass
TC_RISK_003	Verify multiple findings compound risk	Compare single vs multiple detections	None	1. compute_risk with 1 plaque 2. compute_risk with 3 plaques 3. Compare scores	Single plaque@0.9 vs three plaque@0.9	multi.score > single.score	None	multi > single	Pass
TC_RISK_004	Verify empty detections score LOW	Compute risk for empty list	None	1. Call compute_risk with empty list	[]	score=0, level='LOW'	None	score=0, level=LOW	Pass
TC_KNN_001	Verify healthy teeth yield only general advice	Prevention plan for healthy detections	KNN model trained	1. Call generate_prevention_plan with healthy_tooth 2. Observe plan	healthy_tooth@0.9, age=30	Plan has healthy_tooth rec + general advice; no urgent/cavity text	None	No cavity text in plan	Pass
TC_KNN_002	Verify HIGH tier triggers URGENT banner	Prevention plan for severe findings	KNN model trained	1. Call generate_prevention_plan with multiple severe findings 2. Observe first item	2x cavity@0.95 + plaque@0.95, age=30	First plan item is URGENT banner	None	URGENT in plan[0]	Pass
TC_KNN_003	Verify plan orders findings by severity	Check ordering of prevention recommendations	KNN model trained	1. Call generate_prevention_plan with cavity + plaque 2. Check order	plaque@0.8 + cavity@0.8, age=30	cavity text appears before plaque text	None	cavity before plaque	Pass
TC_KNN_004	Verify labeled case journal roundtrip	Append and load clinician-confirmed tiers	None (temp directory)	1. append_labeled_case twice 2. load_labeled_cases 3. Verify count and tiers	6-feature vectors, tiers 2 and 0	load returns 2 rows; tiers match [2, 0]	Journal file created	2 rows, tiers=[2,0]	Pass
TC_KNN_005	Verify retrain blends synthetic + real cases	Retrain model with labeled data	None (temp directory)	1. append_labeled_case 2. retrain_with_labeled 3. Load artifact	6-feature vector, tier 2	Artifact written; model.n_features_in_ == 6	Model artifact updated	n_features_in_ == 6	Pass
TC_KNN_006	Verify confirm-tier endpoint records label	Dentist confirms screening tier	Admin logged in; screening seeded	1. PATCH /screenings/{id}/tier with risk_level='HIGH' 2. Check journal file	{"risk_level":"HIGH"}	200 OK; labeled case appended to journal	Journal file updated	200; journal non-empty	Pass
TC_EVAL_001	Verify eval harness reports holdout accuracy	Run evaluation on synthetic holdout	KNN model trained on bootstrap	1. python -m app.ml.evaluate_knn 2. Check accuracy	Holdout 2000 samples, seed 999	accuracy > 0.85; f1_weighted > 0.80	Report printed to stdout	accuracy=0.9275, f1=0.9251	Pass
TC_EVAL_002	Verify harness reports 0 real cases when journal empty	Check real-case reporting before any reviews	No labeled cases appended	1. run_evaluation() 2. Check real_cases	Empty labeled_cases.jsonl	real_cases.n_real_cases == 0	None	n_real_cases=0	Pass

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_PORTAL_001	Verify patient can view own screening history	Patient logs in and views screenings	Patient account exists; >= 1 screening	1. Login as patient 2. Navigate to portal 3. Observe screening list	Patient JWT	Screening list displayed with risk levels and dates	Toothwise Patient Portal	Screenings displayed	**Pass**
TC_PORTAL_002	Verify patient cannot view other patient's screening	Cross-patient access is blocked	Two patients exist with separate screenings	1. Login as Patient A 2. Attempt GET /api/screenings/self/{Patient_B_screening_id}	Patient A JWT; Patient B screening ID	404 Not Found (not 403 — hides existence)	Toothwise Patient Portal	404 Not Found	**Pass**
TC_NOTE_001	Verify clinical note is saved and returned	Add note to a screening	Screening exists; dentist logged in	1. POST /api/screenings/{id}/notes 2. Check response	{"body":"Mild gingivitis observed"}	201 Created; body matches input	Note stored in DB	201; body=Mild gingivitis observed	**Pass**
TC_NOTE_002	Verify clinical note requires authentication	Attempt to add note without token	None	1. POST /api/screenings/1/notes without token	No token	401 or 403	Access denied	401 Unauthorized	**Pass**
TC_AUDIT_001	Verify screening creation is logged	Check audit trail after screening	Admin logged in; patient exists	1. Create screening 2. Check audit_log table	Screening creation payload	Audit entry exists with action='screening.create'	Audit log updated	Audit entry present	**Pass**
TC_AUDIT_002	Verify tier confirmation is logged	Check audit trail after tier confirm	Screening exists; admin logged in	1. PATCH /screenings/{id}/tier 2. Check audit_log table	{"risk_level":"HIGH"}	Audit entry exists with action='screening.tier_confirm'	Audit log updated	Audit entry present	**Pass**